

Chapter 10

Cursor in Java

Java Cursor is an object that is used to iterate, traverse, or retrieve a Collection or Stream object's elements one by one.

- Cursors allow sequential access to each element in a collection.
- Some cursors (like `ListIterator`) allow addition, removal, or replacement of elements during traversal.
- Cursors are generic, ensuring type safety and avoiding `ClassCastException`

Types of cursor in java

Java provides three types of cursors, depending on the collection type and the operations supported.

1. Enumeration Cursor

The Enumeration cursor is a legacy cursor, introduced before Java 2 (JDK 1.2). It is used mainly with older classes like `Vector` and `Hashtable`.

Declaration of Enumeration

```
public interface Enumeration<E>
```

Important Methods of Enumeration

- **boolean hasMoreElements():** Returns true if more elements are available.
- **E nextElement():** Returns the next element of the collection.

```
import java.util.*;
```

```
public class GFG{
```

```
    public static void main(String[] args){
```

```

Vector<Integer> v = new Vector<>();

v.add(10);
v.add(20);
v.add(30);

Enumeration<Integer> e = v.elements();

System.out.println("Elements using Enumeration:");
while (e.hasMoreElements()) {
    System.out.println(e.nextElement());
}
}
}

```

Output

Elements using Enumeration:

10

20

30

2. Iterator Cursor

The **Iterator cursor** is the universal cursor introduced in JDK 1.2. It can be used with all classes of the Java Collection Framework such as ArrayList, HashSet, LinkedList, etc.

Declaration of Iterator

```
public interface Iterator<E>
```

Important Methods of Iterator

- **boolean hasNext():** Returns true if more elements exist.
- **E next():** Returns the next element.

void remove(): Removes the current element (optional operation).

-

```
import java.util.*;
```

```
public class GFG{
```

```
    public static void main(String[] args){
```

```
        Collection<String> names = new ArrayList<>();
```

```
        names.add("Alice");
```

```
        names.add("Bob");
```

```
        names.add("Charlie");
```

```
        Iterator<String> itr = names.iterator();
```

```
        System.out.println("Elements using Iterator:");
```

```
        while (itr.hasNext()) {
```

```
            String name = itr.next();
```

```
            System.out.println(name);
```

```
            if (name.equals("Bob")){
```

```
                // safe removal
```

```
                itr.remove();
```

```
            }
```

```
        }
```

```
        System.out.println("After removal: " + names);
```

```
    }
```

```
}
```

Output

Elements using Iterator:

Alice

Bob

Charlie

After removal: [Alice, Charlie]

3. ListIterator Cursor

The **ListIterator** cursor is a bidirectional cursor, introduced in JDK 1.2. It is used only with List implementations such as ArrayList and LinkedList.

Declaration of ListIterator

```
public interface ListIterator<E> extends Iterator<E>
```

Important Methods of ListIterator

- **boolean hasNext():** Checks if next element exists.
- **E next():** Returns the next element.
- **boolean hasPrevious():** Checks if previous element exists.
- **E previous():** Returns the previous element.
- **void remove():** Removes current element.
- **void add(E e):** Inserts new element.
- **void set(E e):** Replaces the last element returned by next() or previous().

```
import java.util.*;
```

```
public class GFG{
```

```
    public static void main(String[] args){
```

```
        List<String> list = new ArrayList<>();
```

```
        list.add("Java");
```

```

list.add("Python");

list.add("C++");

ListIterator<String> li = list.listIterator();

System.out.println("Forward Traversal:");
while (li.hasNext()) {
    System.out.println(li.next());
}

System.out.println("Backward Traversal:");
while (li.hasPrevious()) {
    System.out.println(li.previous());
}

// Modify elements
li = list.listIterator();
while (li.hasNext()) {
    String lang = li.next();
    if (lang.equals("Python")) {
        li.set("Kotlin"); // Replace
    }
}

System.out.println("Modified List: " + list);
}
}

```

Output

```
Forward Traversal:
```

Java

Python

C++

Backward Traversal:

C++

Python

Java

Modified List: [Java, Kotlin, C++]

Enumeration

The [Vector class](#) implements a growable array of objects. It is available in java.util package. It implements the List interface. The Enumeration interface defines the methods by which you can traverse the elements in a collection of objects. Now in order to add elements

Vector Syntax:

```
public class Vector<E> extends AbstractList<E> implements List<E>,
RandomAccess, Cloneable, Serializable
```

[java.util.Enumeration](#) interface is one of the predefined interfaces, whose object is used for retrieving the data from collections framework variable(like [Stack](#), [Vector](#), [HashTable](#), etc.) in a forward direction only and not in the backward direction.

You can use the [Java.util.Vector.addElement\(\)](#) method to append a specified element to the end of this vector by increasing the size of the vector by 1. The functionality of this method is similar to that of the add() method of the Vector class.

Syntax:

```
boolean addElement(Object element)
```

Parameters: This function accepts a single parameter *element* of object type and refers to the element specified by this parameter is appended to the end of the vector.

Return Value: This is a void type method and does not return any value.

Example 1:

```
// Java Program to Iterate Vector using Enumeration
```

```
// Importing Enumeration class
```

```
import java.util.Enumeration;
```

```
// Importing vector class
```

```
import java.util.Vector;
```

```
public class GFG {
```

```
    // Main driver method
```

```
    public static void main(String a[])
```

```
    {
```

```
        // Creating a new vector
```

```
        Vector<String> v = new Vector<String>();
```

```
        // Adding elements to the end
```

```
        v.add("Welcome");
```

```
        v.add("To");
```

```
        v.add("Geeks for");
```

```
        v.add("Geeks");
```

```
        // Creating an object of enum
```

```
        Enumeration<String> en = v.elements();
```

```
while (en.hasMoreElements()) {
```

```
    // Print the elements using enum object
```

```
    // of the elements added in the vector
```

```
    System.out.println(en.nextElement());
```

```
}
```

```
}
```

```
}
```